



CSE 203: Data Structure

Data
Structure

TREE:

What is this?





CSE 203: Data Structure

Data
Structure

TREE:

What is this?





CSE 203: Data Structure

Data
Structure

TREE:

Cropped tree.



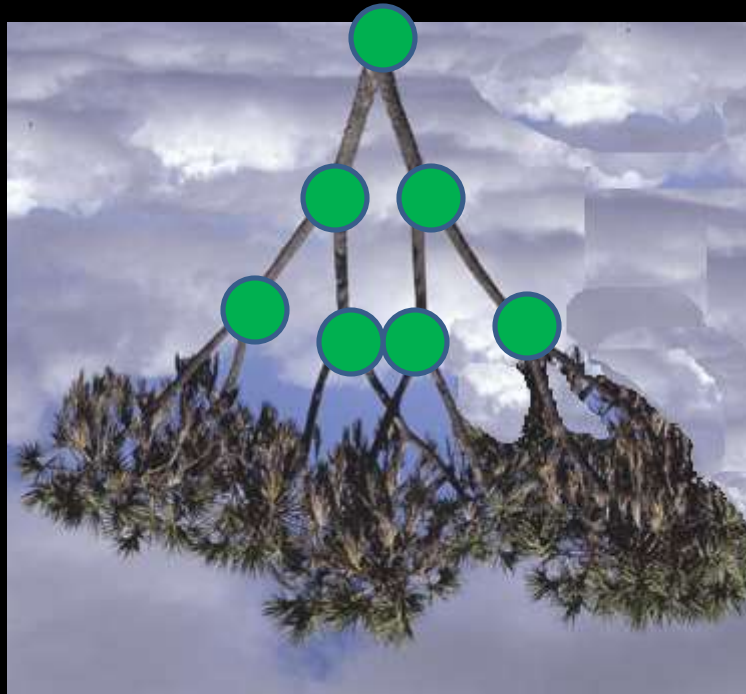


CSE 203: Data Structure

Data
Structure

TREE:

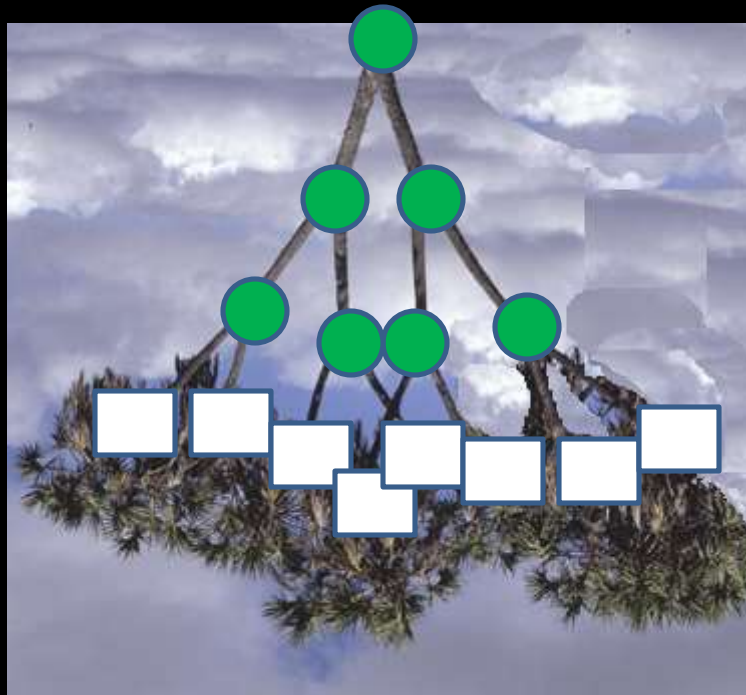
Identify the
branching points.





TREE:

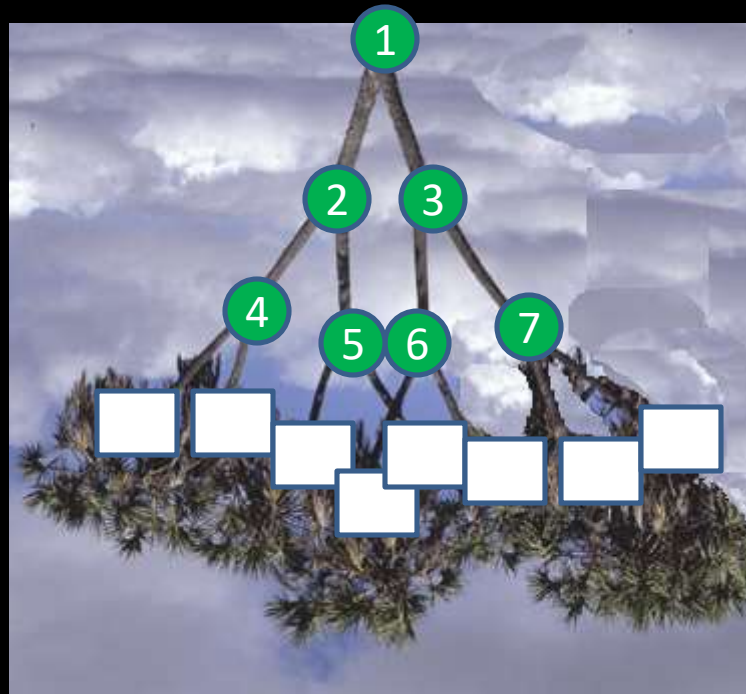
Identify the leaf points.





TREE:

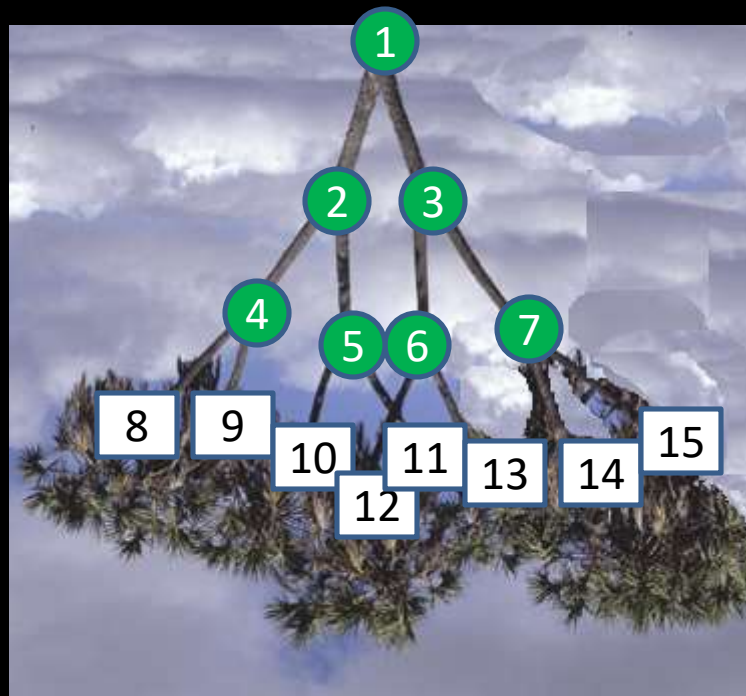
Number each
interior points/
nodes.





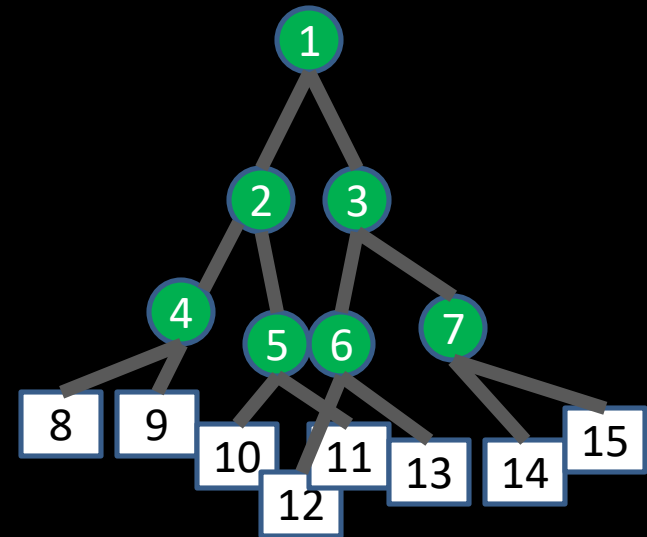
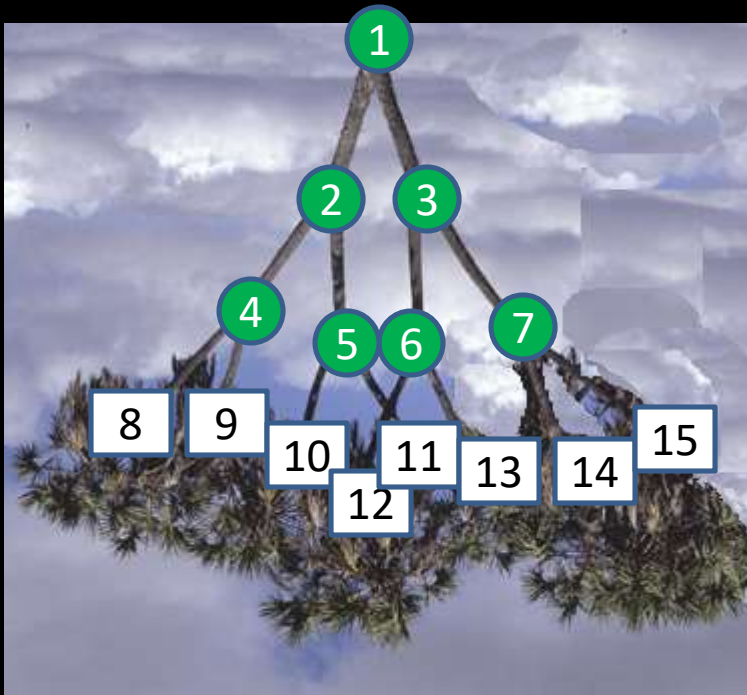
TREE:

Number each leaf
points/ nodes.





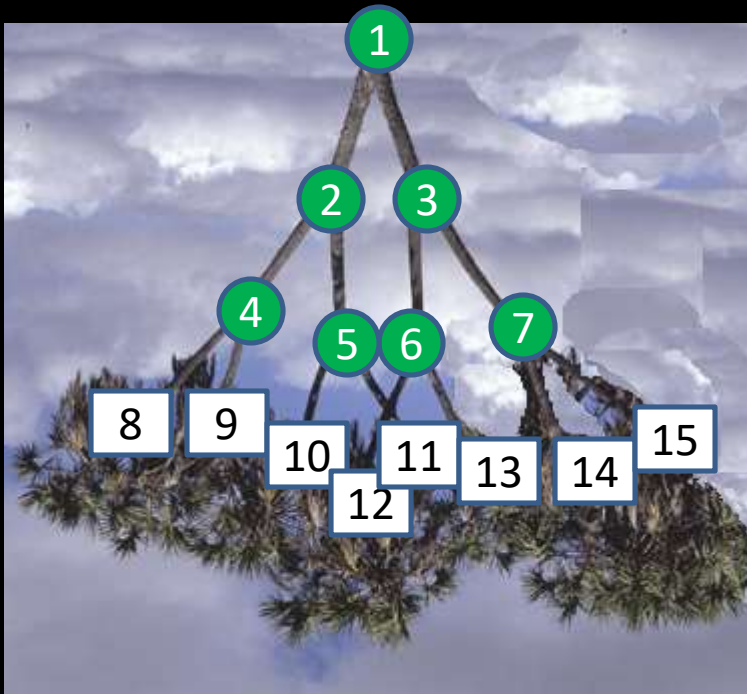
TREE:



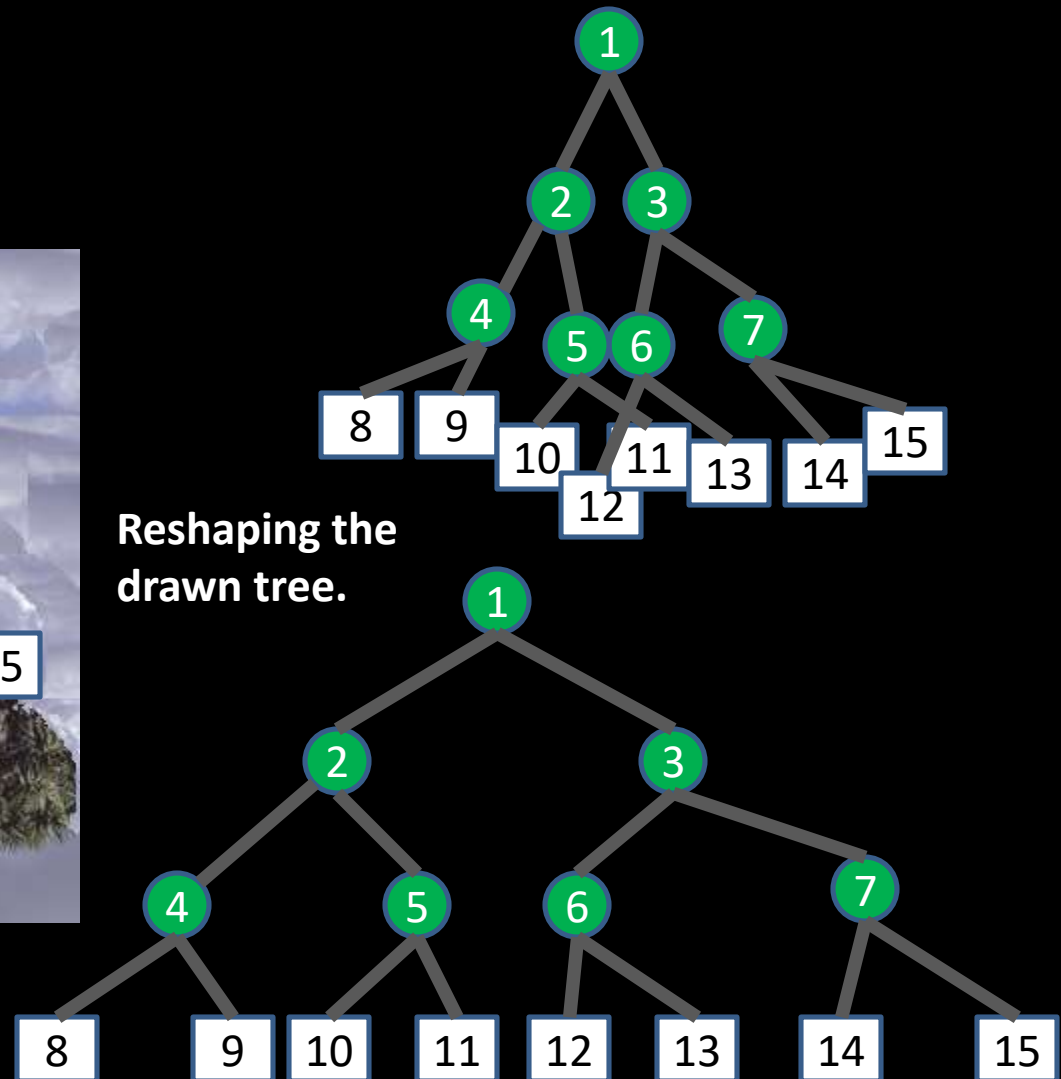
Drawing an instance
of the tree.



TREE:



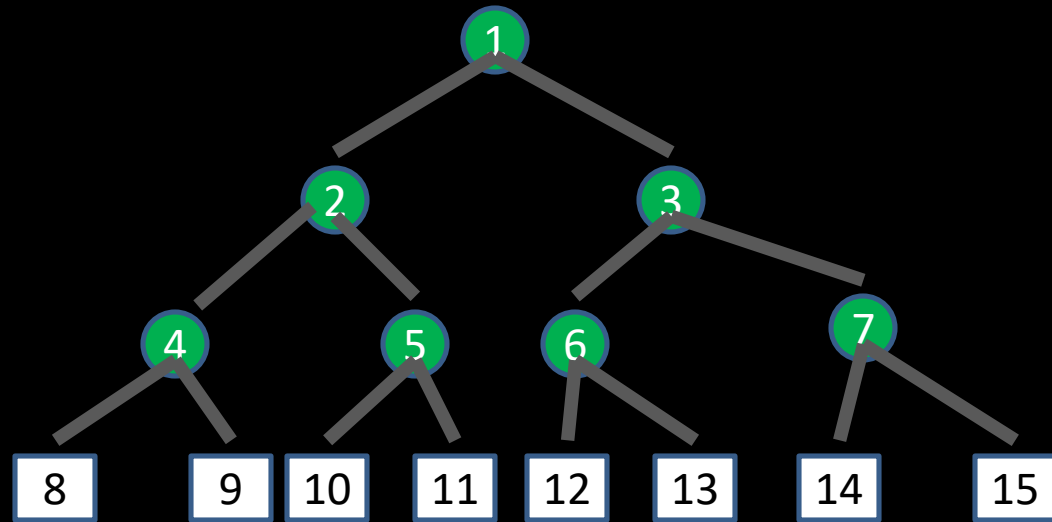
Reshaping the
drawn tree.



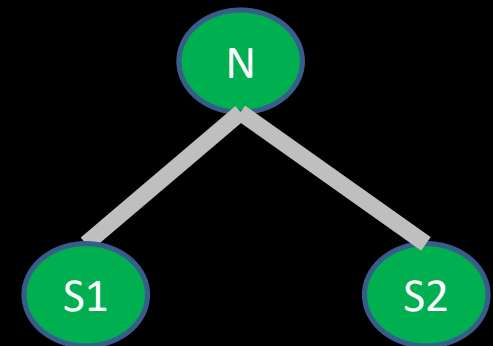


TREE:

This is called tree.



- A tree T has a root node, R
- Each inner node contains one or more successors
- If the root R has successors, each successor branches to a subtree.
- The terminal nodes do not have any successor. These are called leaf nodes.
- The tree T is called empty tree, if its root R is empty.
- If N is a node in T with left successor $S1$ and right successor $S2$,
 - N is called the parent of $S1$ and $S2$.
 - $S1$ and $S2$ are called child of N
 - $S1$ and $S2$ are called siblings.
- Line drawn from a node N to its successor S is called edge.
- A sequence of consecutive edges is called a path.
- A path ending in a leaf is call branch

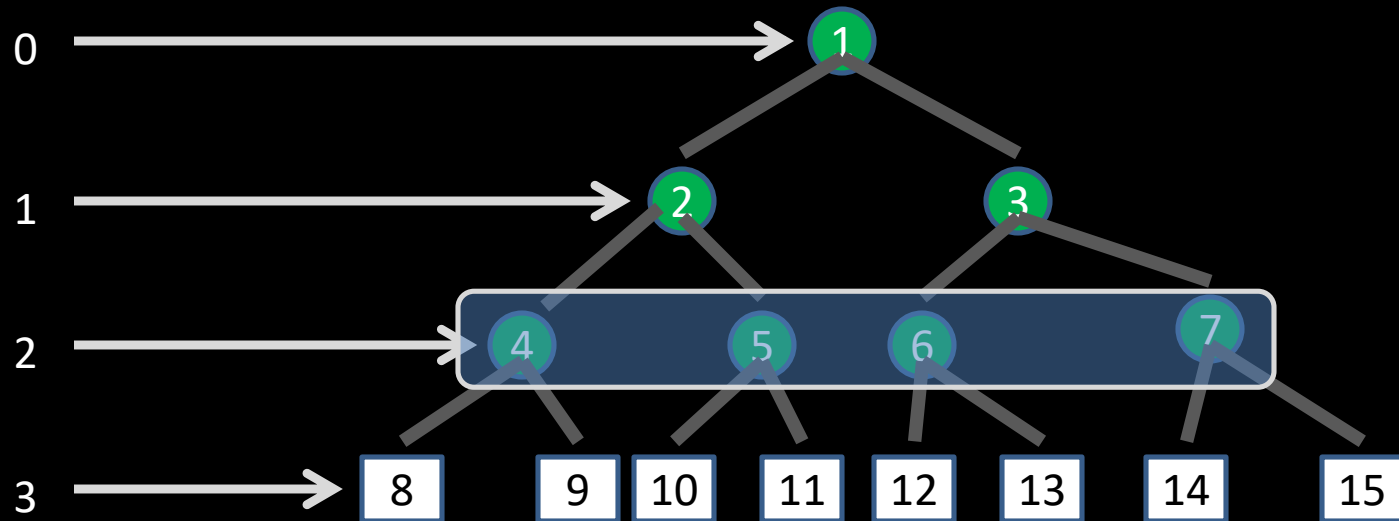




CSE 203: Data Structure

Data
Structure

TREE:



Level of Tree:

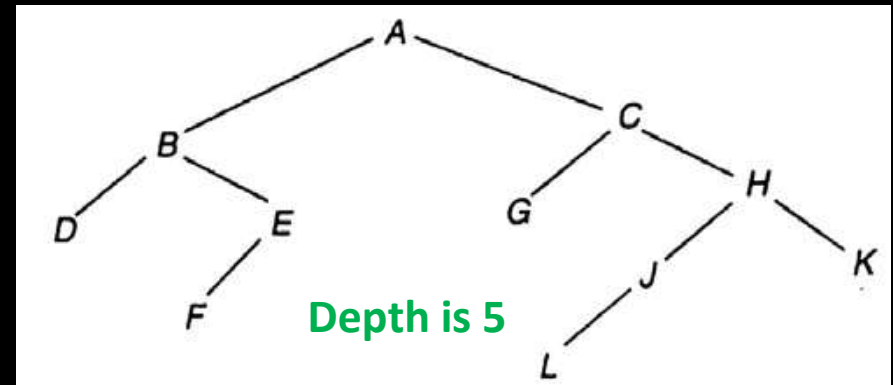
- The root R is assigned a level number 0
- Every node is assigned a level number that is 1 more than its parent.
- The nodes with the same level are called same generation.



TREE:

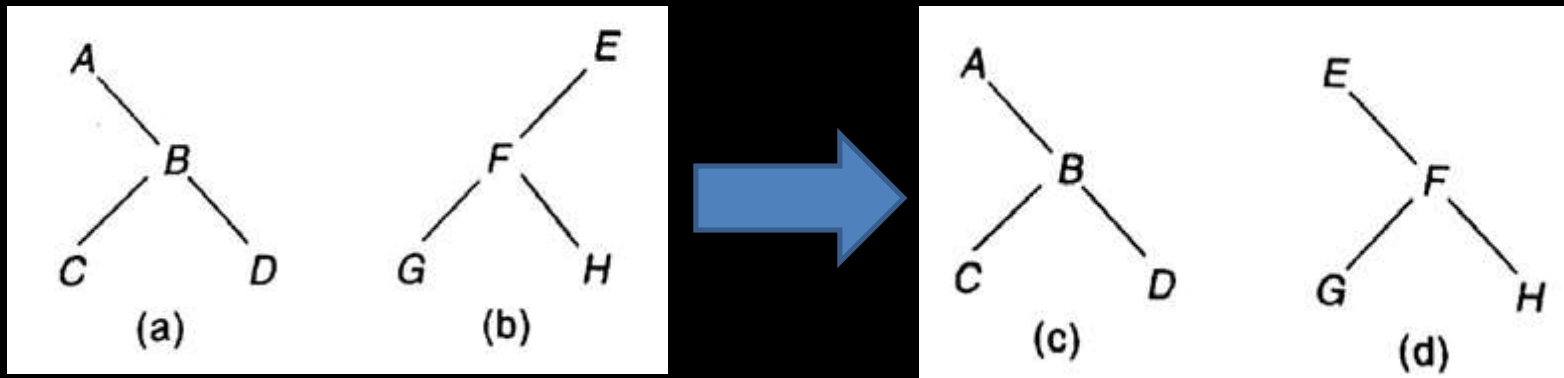
Depth/Height of Tree:

- Maximum number of nodes in a branch.
- It is 1 more than the largest level



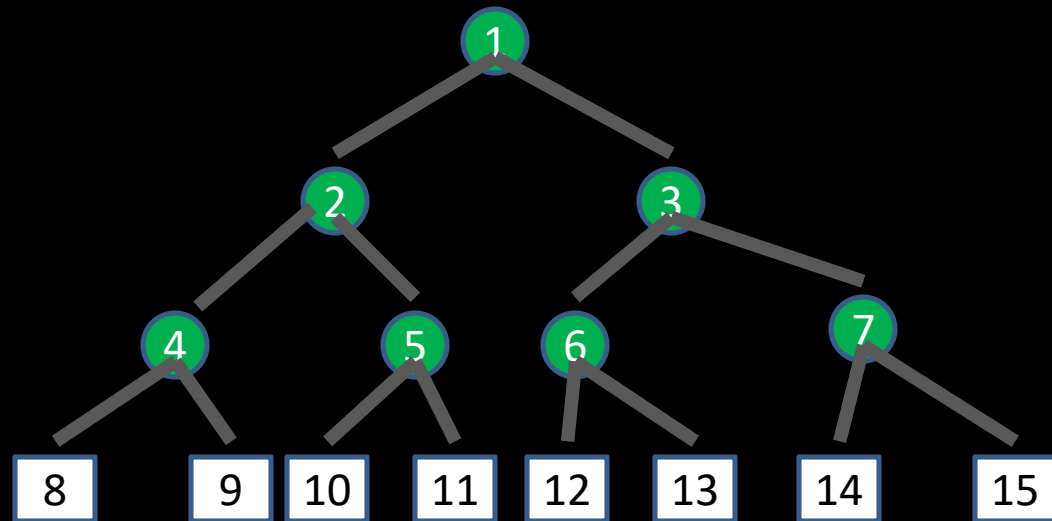
Similar Tree:

Two tree T1 and T2 are called similar if they have the same structure





TREE:



Binary Tree:

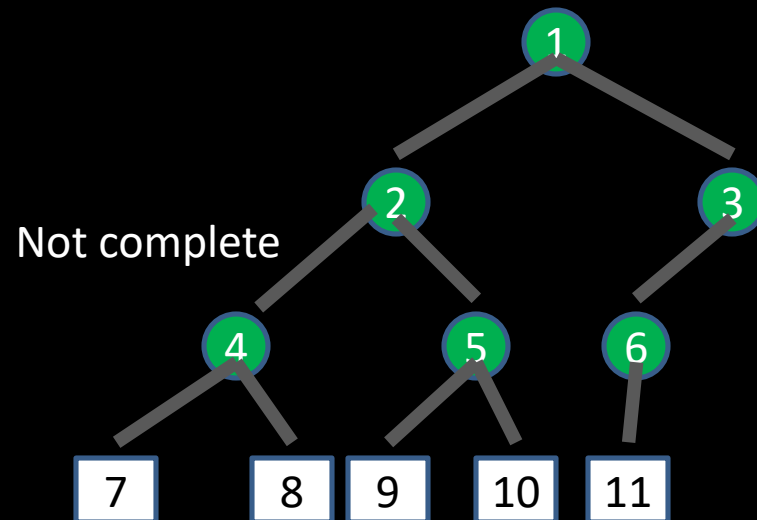
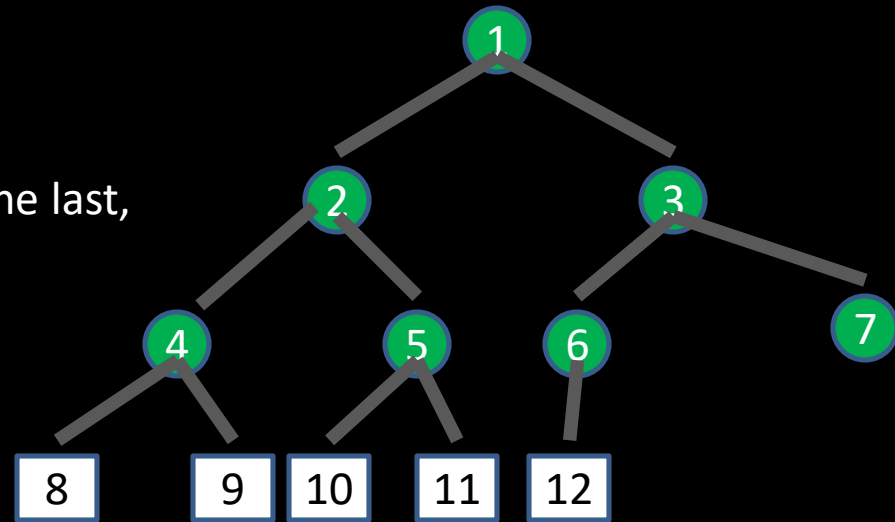
- A binary tree T contains a distinguished node R .
- Each contains no more than two children.
- Remaining nodes of T forms an order pair of disjoint binary tree T_1 and T_2



TREE:

Complete Binary Tree:

- A tree is said complete if all its levels, except the last, have the possible maximum number of nodes.

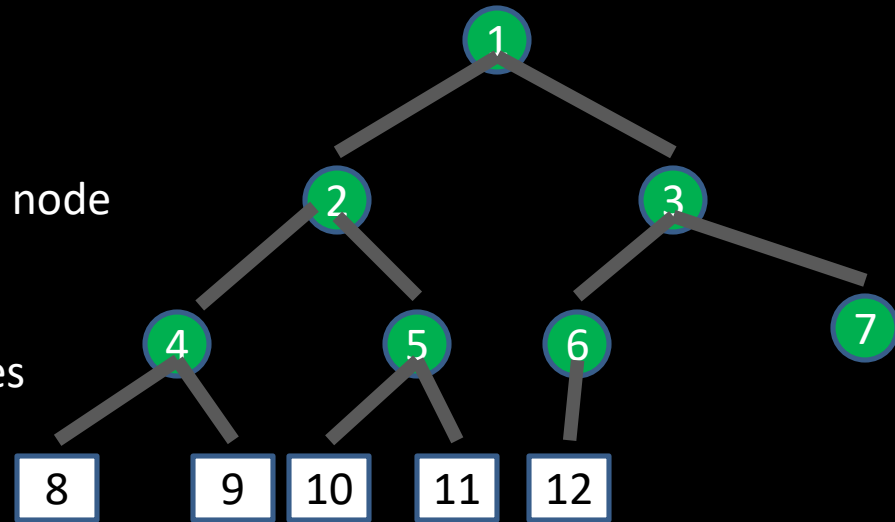




TREE:

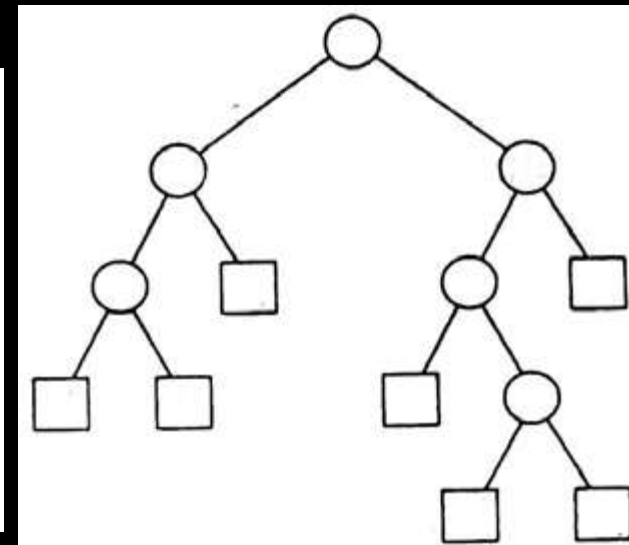
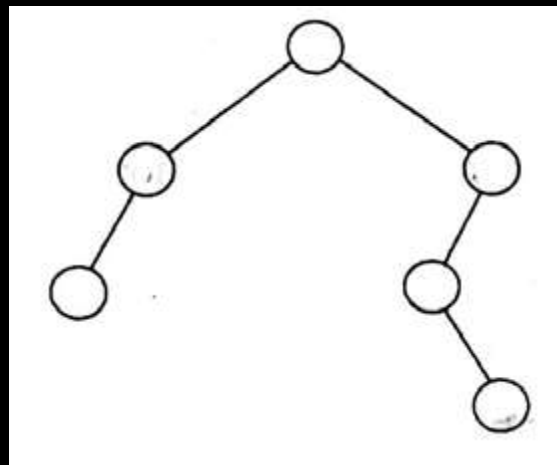
Extended Binary Tree:

- A tree is said extended binary or 2-tree if each node either have 0 or 2 children.
- Nodes with 2 children are called internal node
- Nodes with 0 children are called external nodes
- Internal nodes are represented by circles
- External nodes are represented by rectangles



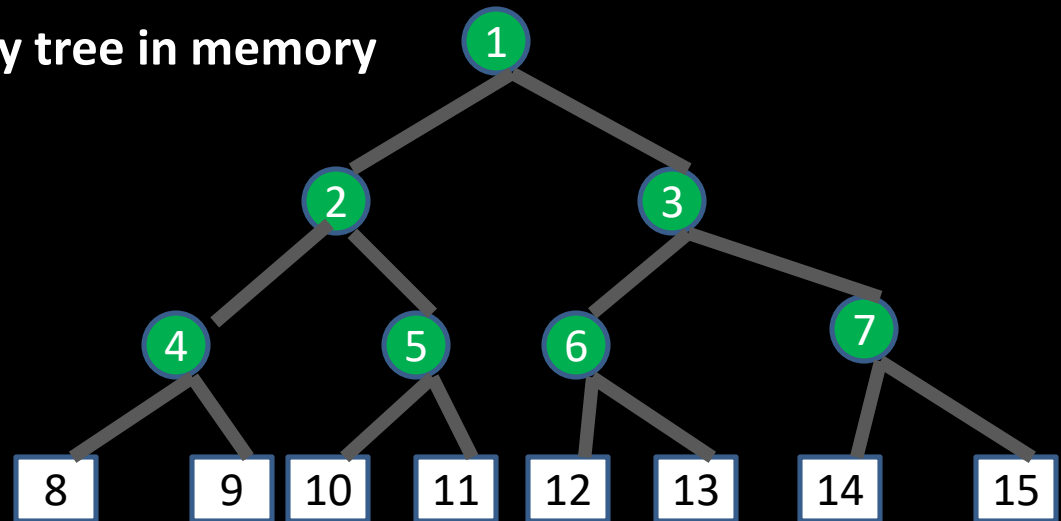
What will happen for such a tree?

Convert it to 2-tree by replacing each empty sub-tree by a new node





TREE: Representation of a binary tree in memory



Link List Representation:

- (1) INFO[K] contains the data at the node N.
- (2) LEFT[K] contains the location of the left child of node N.
- (3) RIGHT[K] contains the location of the right child of node N.



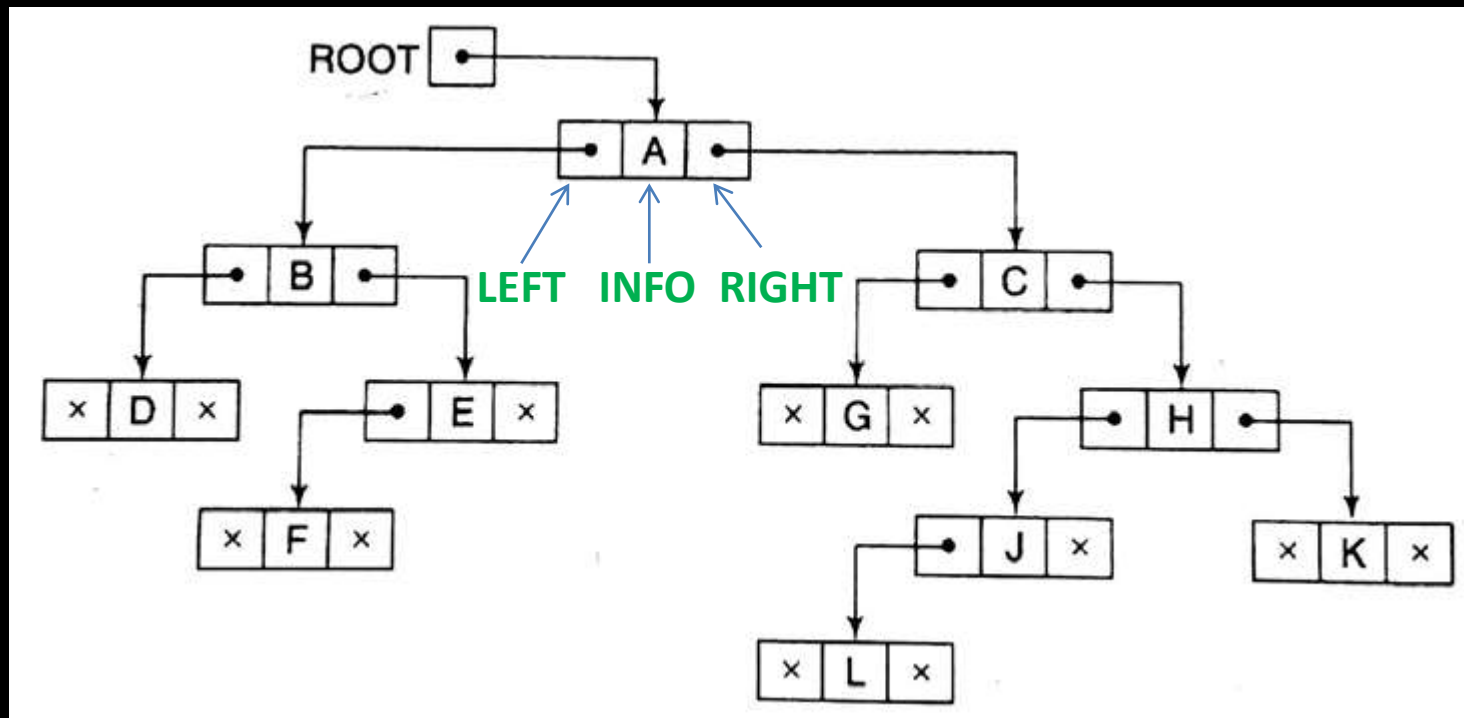
CSE 203: Data Structure

Data
Structure

TREE: Representation of a binary tree in memory

Link List Representation:

- (1) INFO[K] contains the data at the node N.
- (2) LEFT[K] contains the location of the left child of node N.
- (3) RIGHT[K] contains the location of the right child of node N.

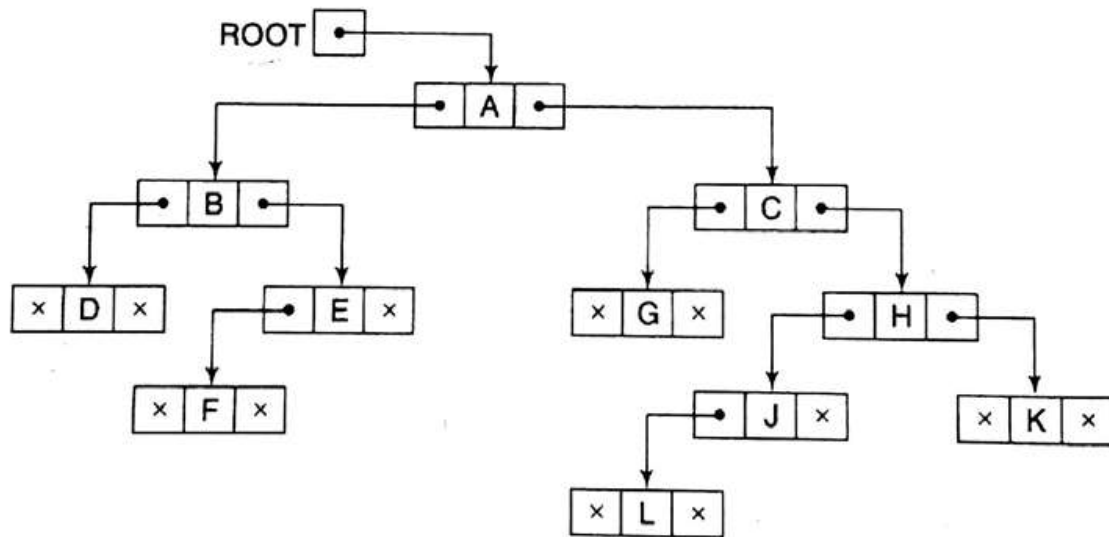




CSE 203: Data Structure

Data
Structure

TREE: Representation of a binary tree in memory Link List Representation:



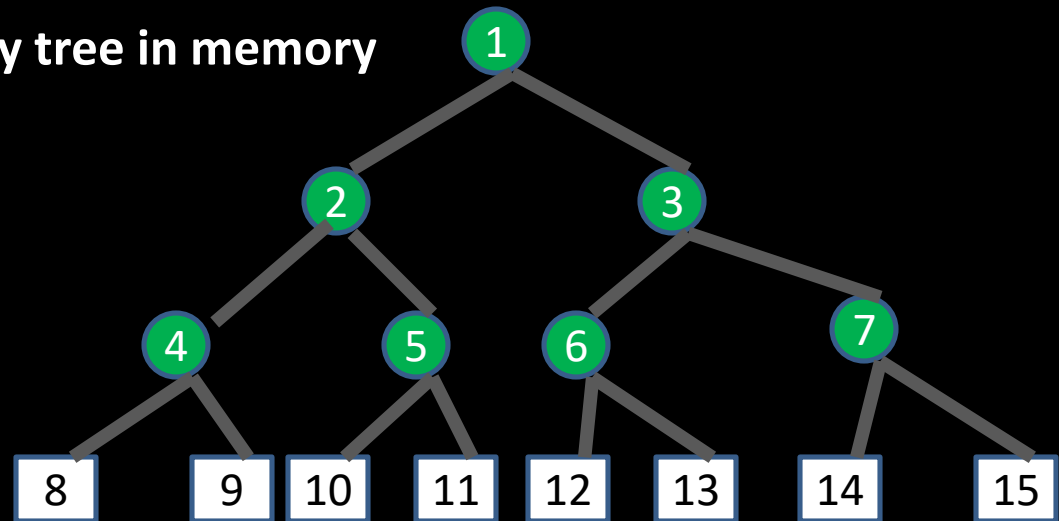
	INFO	LEFT	RIGHT
1	K	0	0
2	C	3	6
3	G	0	0
4		14	
5	A	10	2
6	H	17	1
7	L	0	0
8		9	
9		4	
10	B	18	13
11		19	
12	F	0	0
13	E	12	0
14		15	
15		16	
16		11	
17	J	7	0
18	D	0	0
19		20	
20		0	

Diagram illustrating the Link List Representation of a binary tree. The tree structure is as follows: ROOT points to node A. Node A has a left child B and a right child C. Node B has a left child D and a right child E. Node C has a left child G and a right child H. Node E has a left child F. Node H has a left child J and a right child K. Node J has a left child L. Nodes D, G, L, and K are leaf nodes represented by 'x' in their left and right pointers.

Diagram illustrating the Link List Representation of a binary tree. The tree structure is as follows: ROOT points to node A. Node A has a left child B and a right child C. Node B has a left child D and a right child E. Node C has a left child G and a right child H. Node E has a left child F. Node H has a left child J and a right child K. Node J has a left child L. Nodes D, G, L, and K are leaf nodes represented by 'x' in their left and right pointers.



TREE: Representation of a binary tree in memory



Sequential Representation:

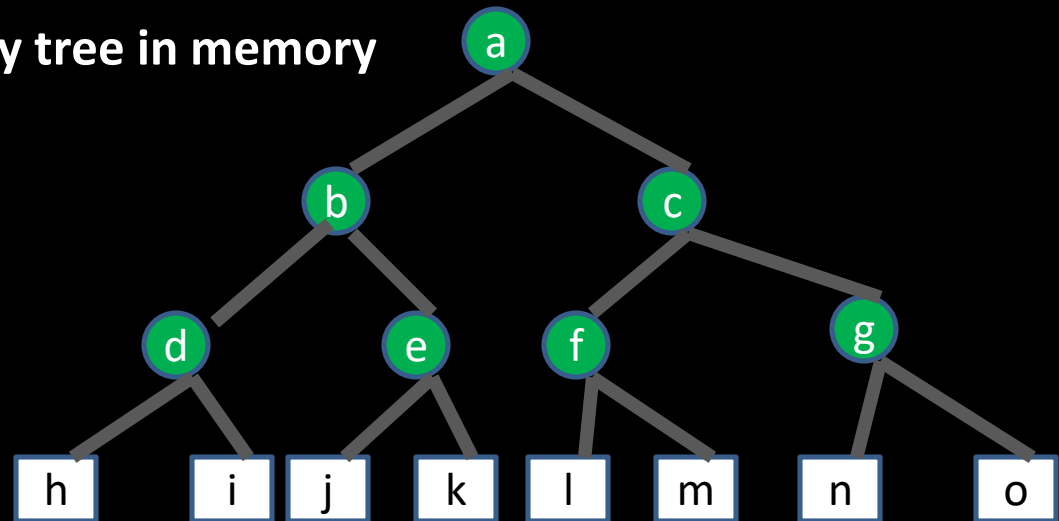
Store items level by level and sequentially in an array

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

- (a) The root R of T is stored in $TREE[1]$.
- (b) If a node N occupies $TREE[K]$, then its left child is stored in $TREE[2 * K]$ and its right child is stored in $TREE[2 * K + 1]$.



TREE: Representation of a binary tree in memory



Sequential Representation:

Store items level by level and sequentially in an array

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- (a) The root R of T is stored in $TREE[1]$.
- (b) If a node N occupies $TREE[K]$, then its left child is stored in $TREE[2 * K]$ and its right child is stored in $TREE[2 * K + 1]$.

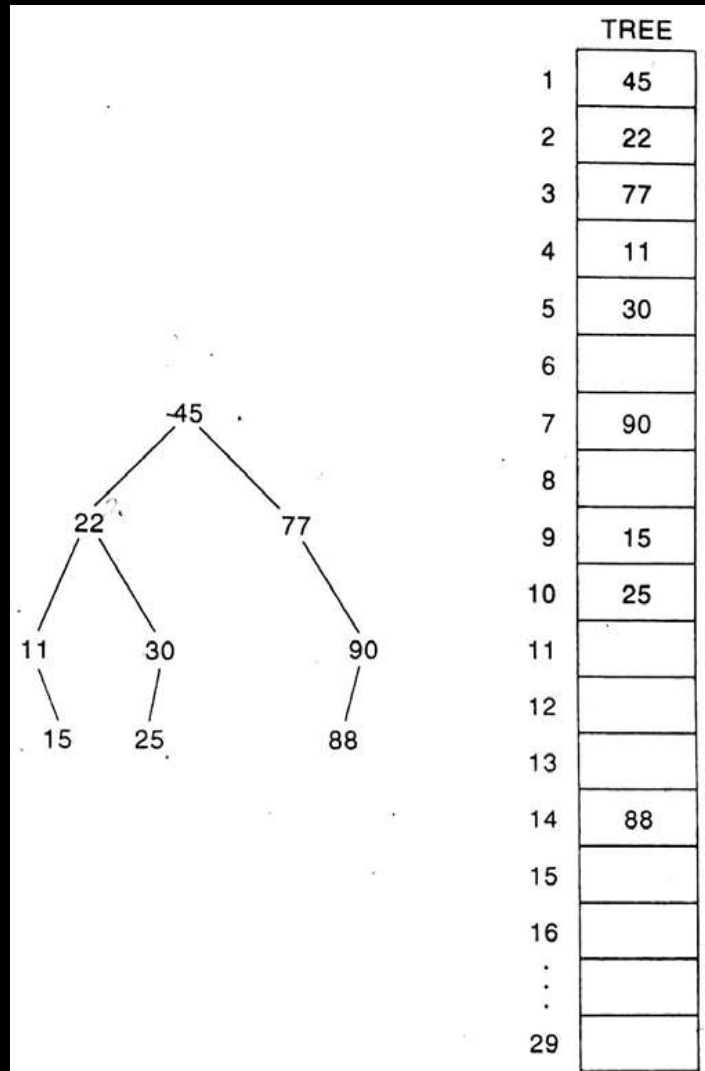


CSE 203: Data Structure

Data
Structure

TREE: Representation of a binary tree in memory

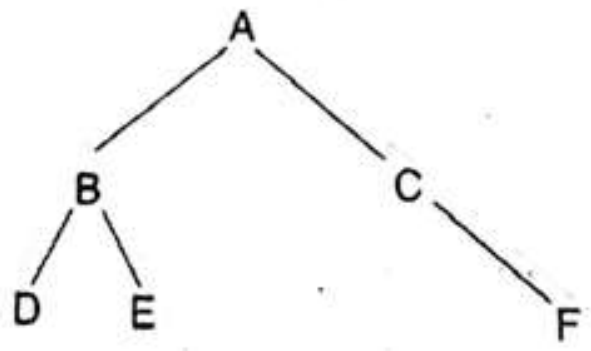
Sequential Representation:





TREE: Traversing

- Preorder
- Inorder
- Postorder



Preorder

- (1) Process the root R.
- (2) Traverse the left subtree of R in preorder.
- (3) Traverse the right subtree of R in preorder.

Preorder: A, B, D, E, C, F

Inorder

- (1) Traverse the left subtree of R in inorder.
- (2) Process the root R.
- (3) Traverse the right subtree of R in inorder.

Inorder: D, B, E, A, C, F

Postorder

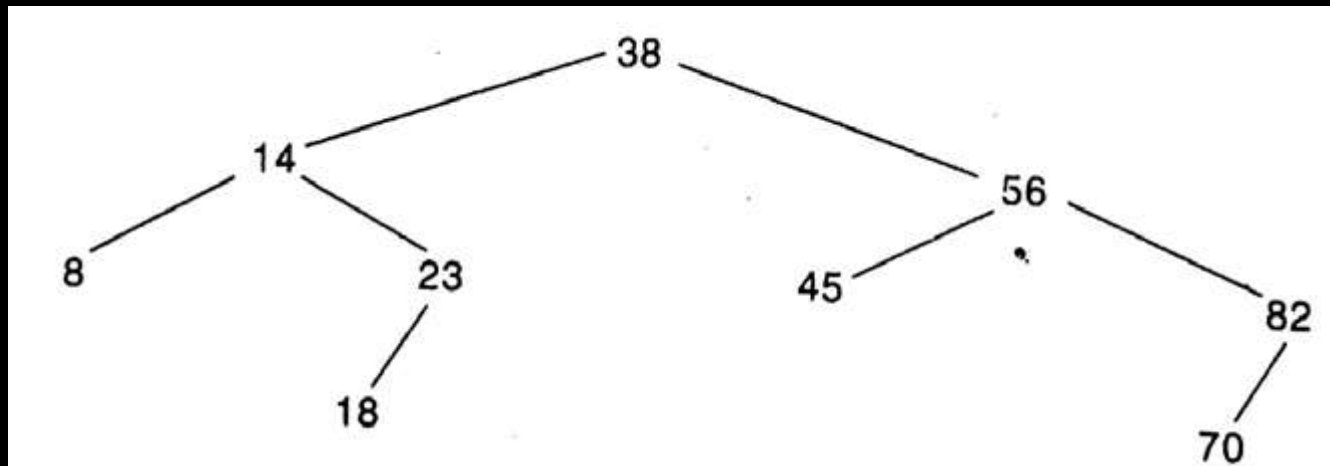
- (1) Traverse the left subtree of R in postorder.
- (2) Traverse the right subtree of R in postorder.
- (3) Process the root R.

Postorder: D, E, B, F, C, A



TREE: Binary Search Tree

A tree T is a binary search tree if each node N contains a value that is greater than the nodes in the left sub-tree and smaller than the nodes in the right sub-tree.





TREE: Binary Search Tree

Items: 40, 60, 50, 33, 55, 11

40

(1) ITEM = 40

40

(2) ITEM = 60

60

40

(3) ITEM = 50

60

50

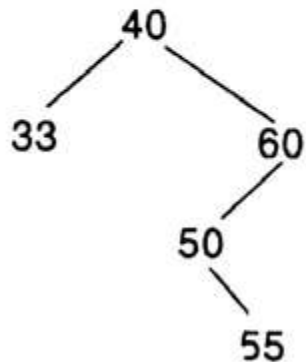
40

33

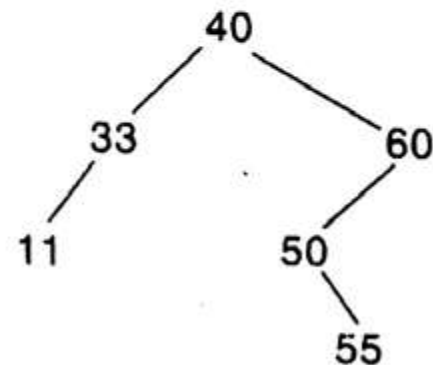
(4) ITEM = 33

60

50



(5) ITEM = 55



(6) ITEM = 11



CSE 203: Data Structure

Data
Structure

TREE:

Huffman's Algorithm:

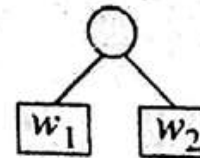
Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \dots, w_n$. Find a tree T' which gives a solution for the $n-1$ weights

$$w_1 + w_2, w_3, w_4, \dots, w_n$$

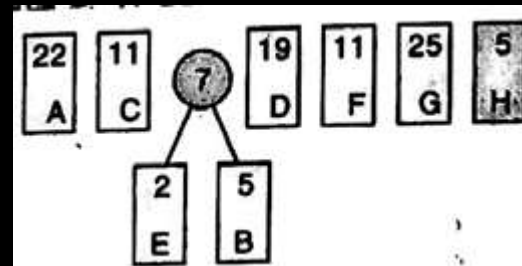
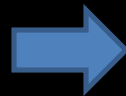
Then, in the tree T' , replace the external node

$w_1 + w_2$

by the subtree



The new 2-tree T is the desired solution.





CSE 203: Data Structure

Data
Structure

TREE:

Huffman's Algorithm:

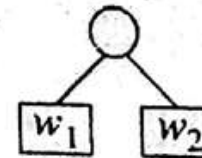
Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \dots, w_n$. Find a tree T' which gives a solution for the $n-1$ weights

$$w_1 + w_2, w_3, w_4, \dots, w_n$$

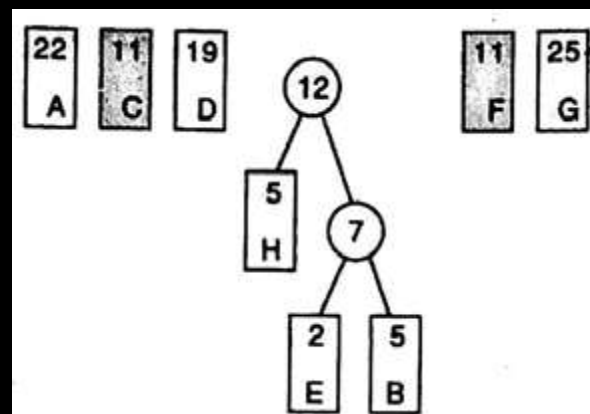
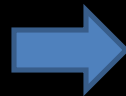
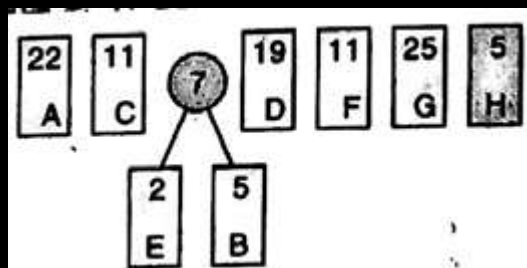
Then, in the tree T' , replace the external node

$w_1 + w_2$

by the subtree



The new 2-tree T is the desired solution.





CSE 203: Data Structure

Data
Structure

TREE:

Huffman's Algorithm:

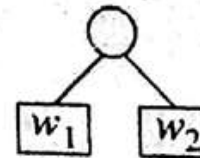
Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \dots, w_n$. Find a tree T' which gives a solution for the $n-1$ weights

$$w_1 + w_2, w_3, w_4, \dots, w_n$$

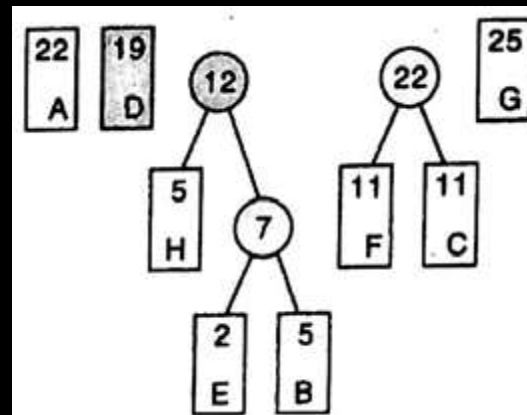
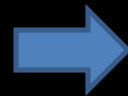
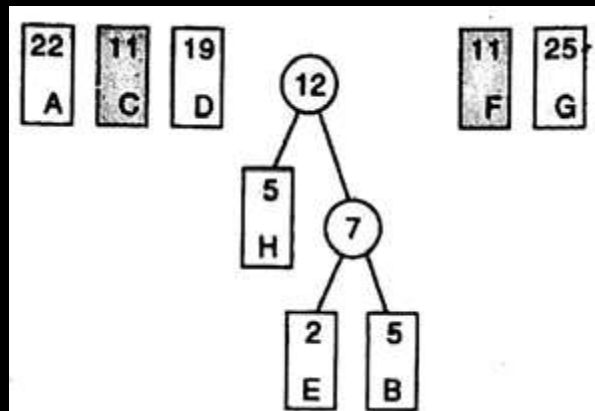
Then, in the tree T' , replace the external node

$w_1 + w_2$

by the subtree



The new 2-tree T is the desired solution.





CSE 203: Data Structure

Data
Structure

TREE:

Huffman's Algorithm:

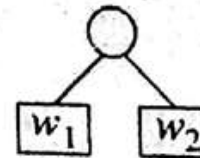
Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \dots, w_n$. Find a tree T' which gives a solution for the $n-1$ weights

$$w_1 + w_2, w_3, w_4, \dots, w_n$$

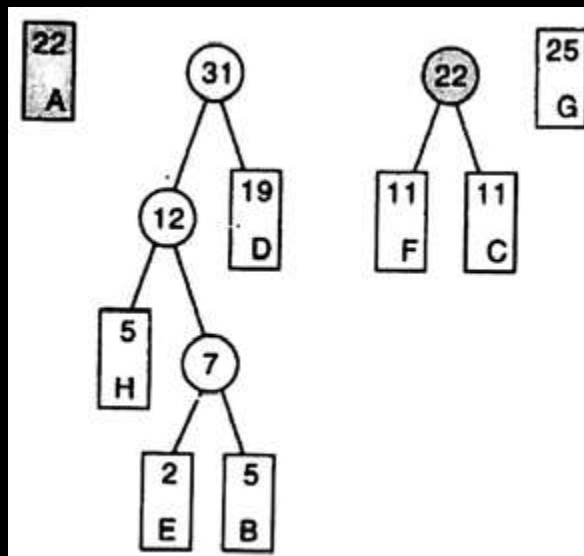
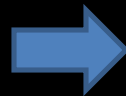
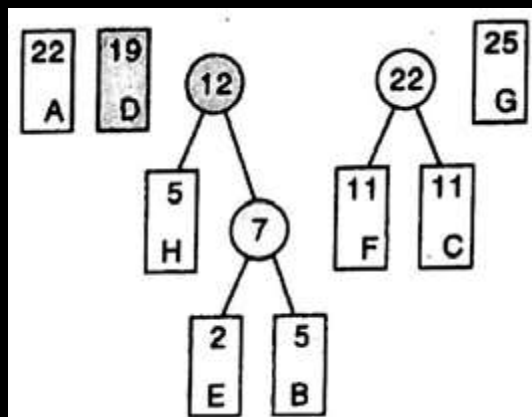
Then, in the tree T' , replace the external node

$w_1 + w_2$

by the subtree



The new 2-tree T is the desired solution.





CSE 203: Data Structure

Data
Structure

TREE:

Huffman's Algorithm:

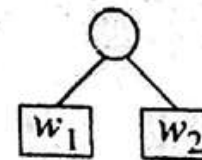
Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \dots, w_n$. Find a tree T' which gives a solution for the $n-1$ weights

$$w_1 + w_2, w_3, w_4, \dots, w_n$$

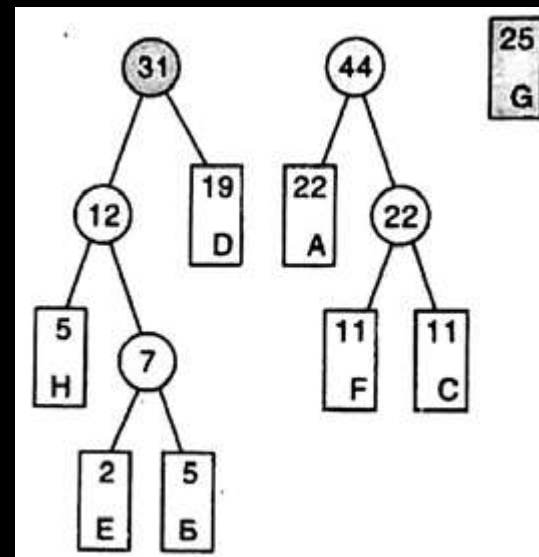
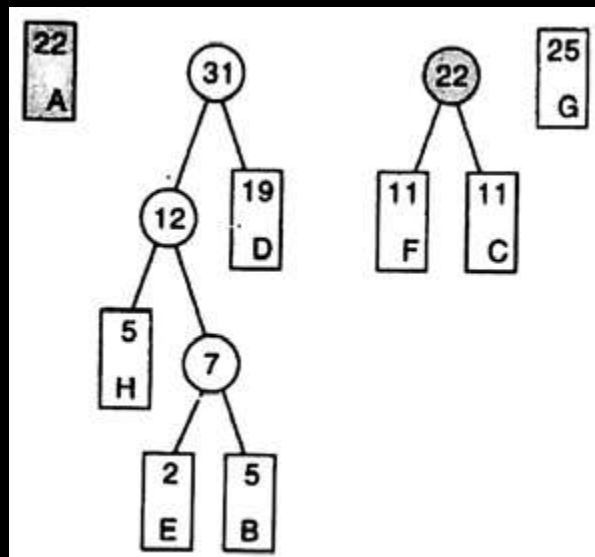
Then, in the tree T' , replace the external node

$w_1 + w_2$

by the subtree



The new 2-tree T is the desired solution.





CSE 203: Data Structure

Data
Structure

TREE:

Huffman's Algorithm:

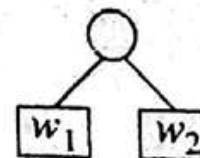
Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \dots, w_n$. Find a tree T' which gives a solution for the $n-1$ weights

$$w_1 + w_2, w_3, w_4, \dots, w_n$$

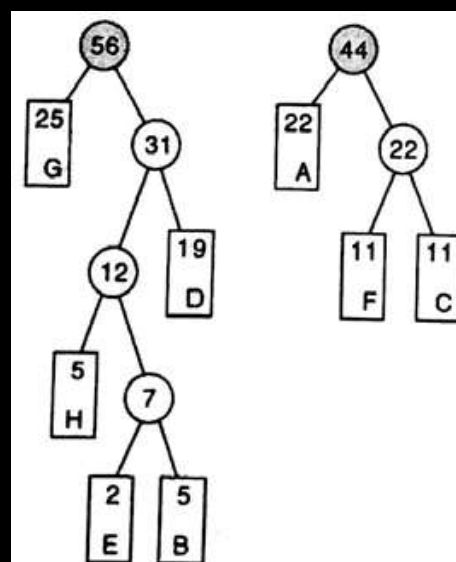
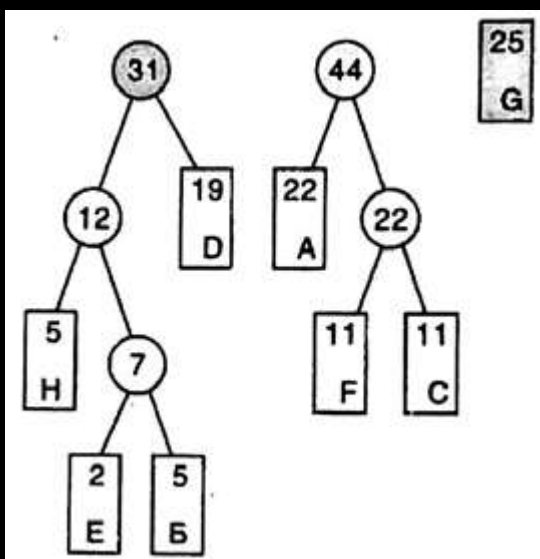
Then, in the tree T' , replace the external node

$w_1 + w_2$

by the subtree



The new 2-tree T is the desired solution.





CSE 203: Data Structure

Data Structure

TREE:

Huffman's Algorithm:

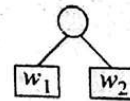
Suppose w_1 and w_2 are two minimum weights among the n given weights $w_1, w_2, \dots, w_n$. Find a tree T' which gives a solution for the $n-1$ weights

$$w_1 + w_2, w_3, w_4, \dots, w_n$$

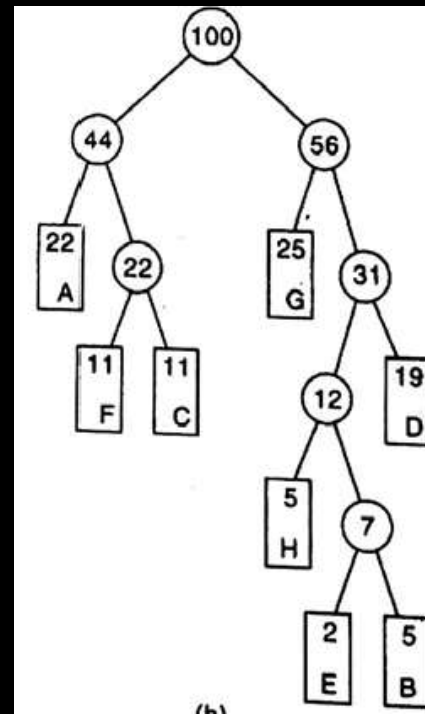
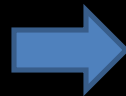
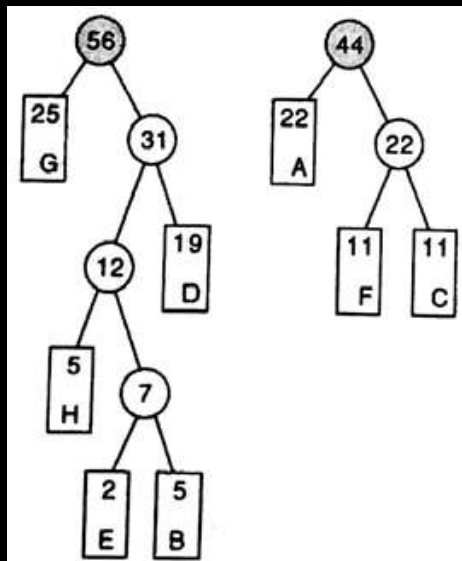
Then, in the tree T' , replace the external node

$w_1 + w_2$

by the subtree



The new 2-tree T is the desired solution.



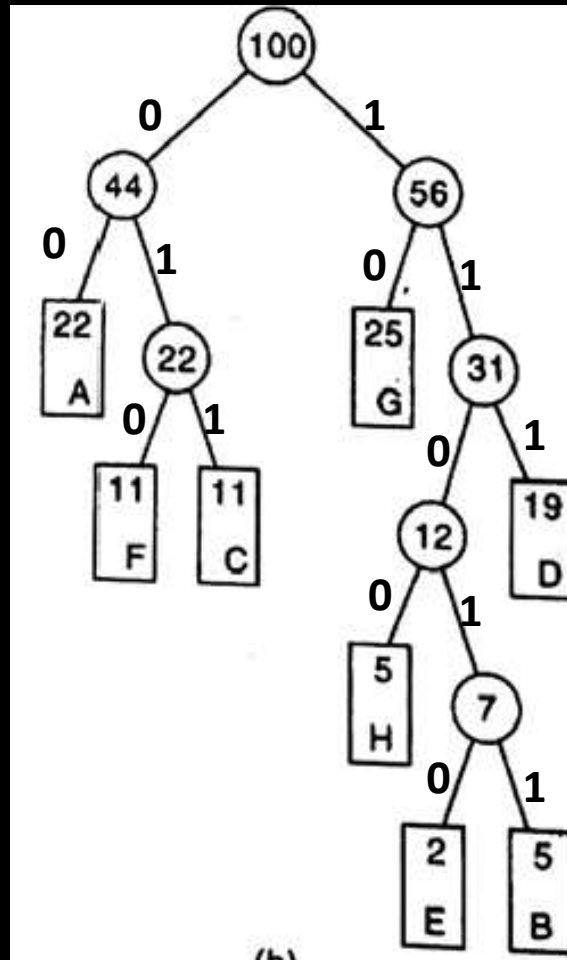
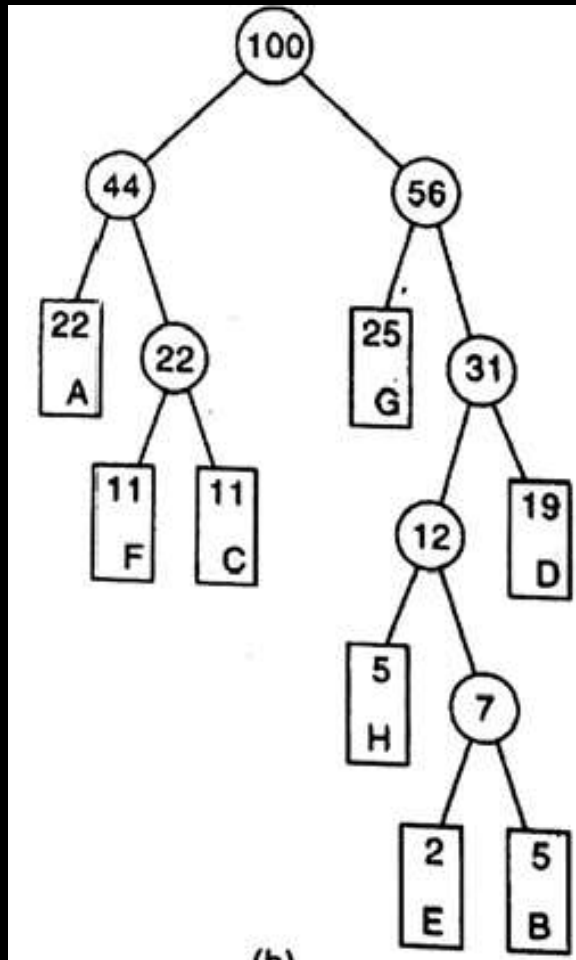


CSE 203: Data Structure

Data
Structure

TREE: Application to coding

Label each edge to a left child by 0
Label each edge to a right child by 1



Huffman's Code:

Traverse from root to leaf
and print the traversed
edge labels

Example:

A: 00

B: 11011

C: 011

D: 111

E: 11010

F: 010

G: 10

H: 1100